Progress Report (As of 03/05/2026)

Plan of Action:

Deadline: June 10, 2026

- Week 0 (1-3 May): Test on fail-stop handler, begin fsync hooking, mc-thread, locking implementation. Paper writing and literature review (parallel).
- Week 1 (4-9 May): syscall hooks close/exit, init, checker. Paper writing and literature review (parallel).
- Week 2 (11-16 May): End to end test using Azure Intel SGX VMs (DCsv2, DCsv3, DCdsv3 series), and Benchmarking. Test includes Threat Modelling, Rollback-attack Scenario, normal test cases, edge cases). Paper writing (parallel).
- Week 3 (18-23 May): Working on future work or novelty work. Paper writing (parallel).
- Week 4 (25-30 June): UAS week. Revalidate future work and E2E test on SGX VM. Paper writing (parallel).
- Week 5 (1-6 June) UAS week. Paper writing and polishing.
- Week 6 (8-10 June) Deadline week, polishing paper.

Progress so far:

1. Code stubs and headers - done✅
2. Monotonic counter (simulated) - done (incl. testing)✅
3. Vault file - done (incl. testing)✅
4. Tag implementation - done (incl. testing)✅
5. Fail-stop handler - done (incl. testing)✅
6. Fsync hook - done (incl. testing)✅
7. MC thread - done (on progress)
8. [Additional] locking implementation (to be implemented, due to the use of global mutex, in which write and fsync hook are required). Idea: mutex implemented on fsync hook or writer, since we want to prevent racecon, where write action will change MAC address mid/during iteration - to be implemented (on progress)
9. Continuing on other syscall hooks e.g. close/exit, and verifying hook timing. - to be implemented ⏳
10. Checker API - to be implemented ⏳
11. E2E test

Additionals:

- Mutex check
- Enqueue
- Crisp on drain
- Checker API actual implementation

Test Results:

1. Monotonic Counter

Files Related: crisp_mc.c
Created 3 states:

- Mc_value: current MC counter (in-memory cache)
- Mc_mu: mutex protecting mc_value access
- mc_mu_initialized: guard so that create_lock only able to be called once

Created mc_uri helper function since PAL API's need URI prefix
Created crisp_mc_init to open existing MC file or create new
Created crisp_mc_read to copy value under lock
Created crisp_mc_increment as an increment counter and persists

Test Case #1 - Fresh run and Increment ✅

```
tjio3fei4@LAPTOP-22NEHGOF:~/gramine$ cd ~/gramine/CI-Examples/intercept && gramine-direct main
[P1:T1:main] Sim MC test start
[P1:T1:main] after init: v=0
[P1:T1:main] after inc1: v=1
[P1:T1:main] after inc2: v=2
[P1:T1:main] after inc3: v=3
[P1:T1:main] Sim MC test done
[P1:T1:main] >>> INTERCEPT: close(fd=3) dipanggil!
[P1:T1:main] >>> INTERCEPT: fsync(fd=3) dipanggil!
[P1:T1:main] >>> INTERCEPT: close(fd=3) dipanggil!
hai hai fasilkom
done
[P1:T1:main] >>> INTERCEPT: exit_group(error_code=0) dipanggil!
tjio3fei4@LAPTOP-22NEHGOF:~/gramine/CI-Examples/intercept$
```

Insight:
To prove that MC File can be created from scratch (value 0), and can bi incremented monotonically 3 times sequentially.

Test Case #2 - Check whether file actually exists in Disk ✅

```
tjio3fei4@LAPTOP-22NEHGOF:~/gramine/CI-Examples/intercept$ ls -la /tmp/crisp_mc.dat
-rw------- 1 tjio3fei4 tjio3fei4 8 May  4 13:09 /tmp/crisp_mc.dat
tjio3fei4@LAPTOP-22NEHGOF:~/gramine/CI-Examples/intercept$ xxd /tmp/crisp_mc.dat
00000000: 0300 0000 0000 0000                      ........
tjio3fei4@LAPTOP-22NEHGOF:~/gramine/CI-Examples/intercept$
```

Insight:
Proven that MC actually stored in disk (not only memory). File size matched (8 byte) and the content matched the last increment (3).

Test Case #3 - Check if the MC actually persists across restarts ✅

```
tjio3fei4@LAPTOP-22NEHGOF:~/gramine/CI-Examples/intercept$ cd ~/gramine/CI-Examples/intercept && gramine-direct main
[P1:T1:main] Sim MC test start
[P1:T1:main] after init: v=3
[P1:T1:main] after inc1: v=4
[P1:T1:main] after inc2: v=5
[P1:T1:main] after inc3: v=6
[P1:T1:main] Sim MC test done
[P1:T1:main] >>> INTERCEPT: close(fd=3) dipanggil!
[P1:T1:main] >>> INTERCEPT: fsync(fd=3) dipanggil!
[P1:T1:main] >>> INTERCEPT: close(fd=3) dipanggil!
hai hai fasilkom
done
[P1:T1:main] >>> INTERCEPT: exit_group(error_code=0) dipanggil!
tjio3fei4@LAPTOP-22NEHGOF:~/gramine/CI-Examples/intercept$ cd ~/gramine/CI-Examples/intercept && gramine-direct main
[P1:T1:main] Sim MC test start
[P1:T1:main] after init: v=6
[P1:T1:main] after inc1: v=7
[P1:T1:main] after inc2: v=8
[P1:T1:main] after inc3: v=9
[P1:T1:main] Sim MC test done
[P1:T1:main] >>> INTERCEPT: close(fd=3) dipanggil!
[P1:T1:main] >>> INTERCEPT: fsync(fd=3) dipanggil!
[P1:T1:main] >>> INTERCEPT: close(fd=3) dipanggil!
hai hai fasilkom
done
[P1:T1:main] >>> INTERCEPT: exit_group(error_code=0) dipanggil!
tjio3fei4@LAPTOP-22NEHGOF:~/gramine/CI-Examples/intercept$
```

Insight:
Persistence worked, MC value still valid when program restarts. Init mus load the value from the disk, not resetting it to 0. So from this test, it can be inferred that the current implementation survives crash/restart.

Test Case #3 - Error Handling (Fail Closed) ✅

```
tjio3fei4@LAPTOP-22NEHGOF:~/gramine/CI-Examples/intercept$ chmod 000 /tmp/crisp_mc.dat
tjio3fei4@LAPTOP-22NEHGOF:~/gramine/CI-Examples/intercept$ cd ~/gramine/CI-Examples/intercept && gramine-direct main
[P1:T1:main] Sim MC test start
[P1:T1:main] error: crisp_mc_init: open failed: -6
[P1:T1:main] Sim MC test done
[P1:T1:main] >>> INTERCEPT: close(fd=3) dipanggil!
[P1:T1:main] >>> INTERCEPT: fsync(fd=3) dipanggil!
[P1:T1:main] >>> INTERCEPT: close(fd=3) dipanggil!
hai hai fasilkom
done
[P1:T1:main] >>> INTERCEPT: exit_group(error_code=0) dipanggil!
tjio3fei4@LAPTOP-22NEHGOF:~/gramine/CI-Examples/intercept$ chmod 600 /tmp/crisp_mc.dat
tjio3fei4@LAPTOP-22NEHGOF:~/gramine/CI-Examples/intercept$
```

Insight:
Tried to block all permission using chmod 000 to make the .dat file inaccessible. It is also proven that the error is “crisp_mc_init: open failed”. This indicates that the current implementation worked as intended as any error does not enforce silent fresh file creation as a fallback.

By this behavior, when the attacker tamper the file permission, the monotonic counter will not silent continue or create new file.

2. Vault File

Files Related:
Implemented files:

- crisp_vault.c
- crisp.h

Gramine LibOS headers:

- libos_handle.h
- libos_fs.h
- libos_lock.h
- libos_internal.h
- linux_abi/fs.h
- linux_abi/errors.h
- And crypto

Gramine internal files:

- libos/src/fs/libos_namei.c
- libos/src/bookkeep/libos_handle.c
- libos/src/fs/libos_fs.c
- libos/src/fs/libos_dcache.c
- libos/src/sys/libos_file.c
- libos/src/sys/libos_open.c

Gramine PF (Protected Files) specific files:

- libos/src/fs/chroot/encrypted.c
- libos/src/fs/libos_fs_encrypted.c
- common/src/protected_files/protected_files.c
- common/src/protected_files/protected_files_format.h

As for testing we added some files on the following files:

- libos/src/libos_init.c for temporary harness test

Some hurdles on implementing Vault File were:

- Cannot use syscall wrapper, for instance we tried libos_syscall_renameat(), its one of the syscall that the app normally use to do rename. But turns out it rejects pointer from LibOS internal memory (return -EFAULT).
- Non recursive lock: Gramine libos_lock is not recursive. On the locking mechanism, the solution was to acquire g_dcache_lock first, but the lookup inside the lock must use the version that is no longer acquired.
- Recursion guard: Because vault writes go through the LibOS VFS (not directly to PAL) to get PF encryption, when mc-thread writes vault, the VFS triggers internal fsync and on future implementation, this will trigger the CRISP fsync hook. crisp_on_fsync enqueue batch, then wake mc-thread, then vault write again causes infinite recursion. The solution was to add g_in_crisp_io flag, a global bool. I originally wanted to use __thread, but Gramine LibOS is linked with -nostdlib (see libos/src/meson.build), __tls_get_addr undefined which can cause link error.
- Since we cannot use syscall wrapper, there is do_rename as an internal helper. But it is static and does not expose to outer file. So I replicated the core logic on crisp_vault.c instead making the do_rename dynamic.

Current implementation:

1. Write to tmp file via get_new_handle, open_namei, do_handle_write. Fs_ops then flush. All are done via LibOS VFS so the PF encryption applies..
2. The rename atomically to swap file with the new file. Why dont we overwrite? Cause it may corrupted the file. So safer alternative is to rename atomically since it renames the file completely and will not cause any file corruption while maintaining file integrity.
3. Recursion guard via plain global bool, reset in all exit path using goto pattern. This is implemented to avoid deadlock or infinite loop on CRISP.

Test Case #1 - Fresh Run (Save and Load) ✅

Insight:

To prove basic vault lifecycle, from fresh install detection, save successm and load that returns Tag and L (same value), and save the second time (atomic update) that does not corrupt the old file.

Test Case #2 - File is in Disk and Encrypted ✅

```
wsl ~/gramine/CI-Examples/crisp-vault-test git:(feat/fail-stop-handler)  6 files changed, 71 insertions(+), 72 deletions(-) (0.05
ls -la vault_dir/vault.dat
-rw------- 1 tjio3fei4 tjio3fei4 4096 May  4 14:02 vault_dir/vault.dat

wsl ~/gramine/CI-Examples/crisp-vault-test git:(feat/fail-stop-handler)  6 files changed, 71 insertions(+), 72 deletions(-) (0.062s
xxd vault_dir/vault.dat | head -3
00000000: 4752 4146 535f 5046 0200 69b0 5a37 4ed7  GRAFS_PF..i.Z7N.
00000010: 80ce fc6a 5e5c 7664 5ff1 2589 ec7b c186  ...j^\vd_.%..{..
00000020: ba61 28bc 6f02 69d6 fa37 ac11 1ef0 1216  .a(.o.i..7......
```

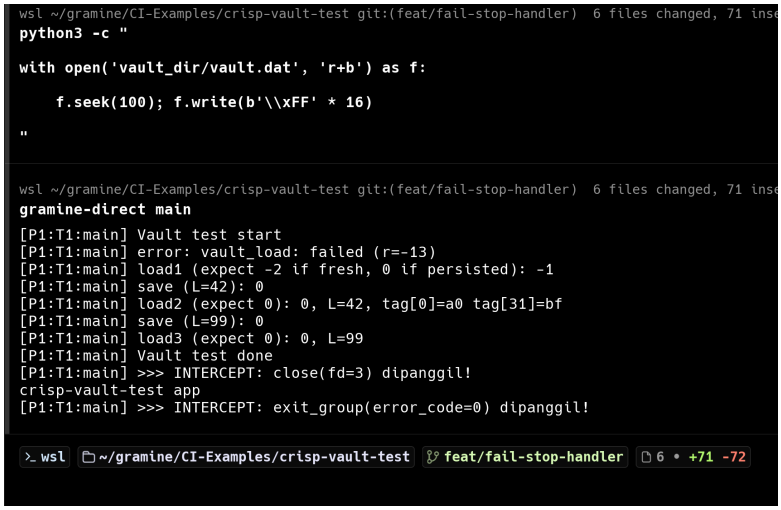
Insight:
To prove that the vault actually encrypted by the Gramine PF layer. So anything outside the enclave file only be seen as ciphertext, not plaintext that leaks tag value and L value. Result confirms that the file has GRAFS_PF header, a Gramine Protected File signature. Means we successfully encrypted the file via PF.

Test Case #3 - Persistence (Re run) ✅

```
wsl ~/gramine/CI-Examples/crisp-vault-test git:(feat/fail-stop-handler)  6 files changed, 7
gramine-direct main
[P1:T1:main] Vault test start
[P1:T1:main] load1 (expect -2 if fresh, 0 if persisted): 0
[P1:T1:main] save (L=42): 0
[P1:T1:main] load2 (expect 0): 0, L=42, tag[0]=a0 tag[31]=bf
[P1:T1:main] save (L=99): 0
[P1:T1:main] load3 (expect 0): 0, L=99
[P1:T1:main] Vault test done
[P1:T1:main] >>> INTERCEPT: close(fd=3) dipanggil!
crisp-vault-test app
[P1:T1:main] >>> INTERCEPT: exit_group(error_code=0) dipanggil!

wsl ~/gramine/CI-Examples/crisp-vault-test git:(feat/fail-stop-handler)  6 files changed, 7
gramine-direct main
[P1:T1:main] Vault test start
[P1:T1:main] load1 (expect -2 if fresh, 0 if persisted): 0
[P1:T1:main] save (L=42): 0
[P1:T1:main] load2 (expect 0): 0, L=42, tag[0]=a0 tag[31]=bf
[P1:T1:main] save (L=99): 0
[P1:T1:main] load3 (expect 0): 0, L=99
[P1:T1:main] Vault test done
[P1:T1:main] >>> INTERCEPT: close(fd=3) dipanggil!
crisp-vault-test app
[P1:T1:main] >>> INTERCEPT: exit_group(error_code=0) dipanggil!
```

Insight:
Since the file existed from the previous test, the load1 output shows 0. And the tag and value persists.

Test Case #4 - File tamper test ✅



Insight:
This test has not been demonstrated during the most recent demo. So result shows that attacker cannot tamper the file without being detected. r=-13 means EACCES from PF layer. Load1 returns -1 which means CRISP process halted. PF layer AES-GCM MAC verification failed and refuses open, then CRISP halt (return -1).

3. Tag

The tag that contains hash that sums up the state of all Protected Files onto one 32-byte value. If file contents altered, tag changes. If an attacker rollback one of the file, the computed tag would be different than the one stored in vault.

Files Related:

- **crisp_tag.c**
    - crisp_compute_global_tag() iterate all PF, and has all MAC
    - crisp_flush_pf_by_path() force flush one PF, used by exit hook so that the MAC is fresh before the tag being computed
    - extract_pf_mac() to open PF, and get MAC, then close.

Note that Gramine does not expose accessor for metadata_mac. So we should independently add 2 new functions on the 2 upstream Gramine files. Files are: protected_files.h, protected_files.c. In here metadata_mac embedded in pf context (not pointer). Plaintext_part is the part of metadata that is not encrypted. metadata_mac is the 16-byte AES-GCM tag that covers the whole file (per-file Merkle root).

Implementations:

- Crisp_compute_global_tag: g_crisp.pf_paths[] must be pre-sorted lexicographically so that the Tag is deterministic, PF order remains the same, and hash will also be the same. If this is not sorted, any separate run sessions will yield different hash.
- Metadata_mac is used instead of the hashing the file content, because metadata_mac is AES-GCM tag that covers all file via PF internal merkle tree. If any byte altered in PF, metadata_mac changes. So we (user side) will get per-file root hash from the PF layer, and no need to hash the whole content.
- Extract_pf_mac uses the following access path: hdl (file handle from open namei), .inode (struct libos_inode), .data (cast to (struct libos_encrypted_file*)), .pf (internal PF layer). .metadata_node, .plaintext_part, .metadata_mac.

Also, implementing inode lock because on libos_fs_encrypted.h, stated that “Operations on a single libos_encrypted_file are NOT thread-safe, it is intended to be protected by a lock”. All existing callers read/write/flush uses lock(&hdl->inode->lock), so we follow the same pattern.

Gramine implementation files:

- common/src/protected_files/protected_files_format.h
- common/src/protected_files/protected_files_internal.h
- libos/include/libos_fs_encrypted.h
- libos/src/fs/chroot/encrypted.c
- libos/src/fs/libos_fs_encrypted.c

This implementation mimics the SCONE FSPF Merkle root, where one value represents the state of all PF. Test code:

```
  GNU nano 6.2                                                  main.c
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>

int main(void) {
    {
        printf("crisp-tag-test: creating PF files\n");
        //printf("crisp-tag-test: READ-ONLY\n");

        const char* paths[] = {"/crisp/a.dat", "/crisp/b.dat"};
        const char* contents[] = {"hello from a", "hello from b"};

        for (int i = 0; i < 2; i++) {
            int fd = open(paths[i], O_WRONLY | O_CREAT | O_TRUNC, 0600);
            //int fd = open(paths[i], O_RDONLY);
            if (fd < 0) {
                printf("  FAIL open %s\n", paths[i]);
                return 1;
            }
            write(fd, contents[i], strlen(contents[i]));
            fsync(fd);
            close(fd);
            printf("  wrote %s\n", paths[i]);
            //char buf[64] = {0};
            //read(fd, buf, sizeof(buf) - 1);
            //close(fd);
            //printf("  read %s: %s\n", paths[i], buf);
        }
    }
    return 0;
}
```

Test Case #1 - Create PF Files, error expected ✅

```
wsl ~/gramine/CI-Examples/crisp-tag-test git:(feat/fail-stop-handler)  6 files changed, 21 insertions(+), 22 deletions(-) (
gramine-direct main
[P1:T1:main] Tag test start
[P1:T1:main] error: tag: extract failed for /crisp/a.dat
[P1:T1:main] tag_boot:    ret=-1 hex=306cfdf7ff7f00003b89fdf7ff7f00000000000000000000d01d000000000000
[P1:T1:main] flush a.dat: ret=-1
[P1:T1:main] flush b.dat: ret=-1
[P1:T1:main] error: tag: extract failed for /crisp/a.dat
[P1:T1:main] tag_after:   hex=20a0eaffff7f00000000000000000000d7f9ffff7f0000b6b9fdf7ff7f0000
[P1:T1:main] same_in_run: NO
[P1:T1:main] Tag test done
[P1:T1:main] >>> INTERCEPT: close(fd=3) dipanggil!
[P1:T1:main] >>> INTERCEPT: fsync(fd=3) dipanggil!
[P1:T1:main] >>> INTERCEPT: close(fd=3) dipanggil!
[P1:T1:main] >>> INTERCEPT: fsync(fd=3) dipanggil!
[P1:T1:main] >>> INTERCEPT: close(fd=3) dipanggil!
crisp-tag-test: creating PF files
  wrote /crisp/a.dat
  wrote /crisp/b.dat
[P1:T1:main] >>> INTERCEPT: exit_group(error_code=0) dipanggil!

 wsl  ~/gramine/CI-Examples/crisp-tag-test  feat/fail-stop-handler  6 • +21 -22
```

Insight:
App creates a new file (a.dat and b.dat).

Test Case #2 - Files exist, tag success ✅

```
wsl ~/gramine/CI-Examples/crisp-tag-test git:(feat/fail-stop-handler)  6 files changed, 21 insertions(+), 22 deletions(-) (0.167s
gramine-direct main
[P1:T1:main] Tag test start
[P1:T1:main] tag_boot:    ret=0 hex=31c5522a3a9d7e357c90e1263260f384d70cb2ca93360b6b82bf4feb3fc91eeb
[P1:T1:main] flush a.dat: ret=0
[P1:T1:main] flush b.dat: ret=0
[P1:T1:main] tag_after:   hex=31c5522a3a9d7e357c90e1263260f384d70cb2ca93360b6b82bf4feb3fc91eeb
[P1:T1:main] same_in_run: YES
[P1:T1:main] Tag test done
[P1:T1:main] >>> INTERCEPT: close(fd=3) dipanggil!
[P1:T1:main] >>> INTERCEPT: fsync(fd=3) dipanggil!
[P1:T1:main] >>> INTERCEPT: close(fd=3) dipanggil!
[P1:T1:main] >>> INTERCEPT: fsync(fd=3) dipanggil!
[P1:T1:main] >>> INTERCEPT: close(fd=3) dipanggil!
crisp-tag-test: creating PF files
  wrote /crisp/a.dat
  wrote /crisp/b.dat
[P1:T1:main] >>> INTERCEPT: exit_group(error_code=0) dipanggil!

 wsl  ~/gramine/CI-Examples/crisp-tag-test  feat/fail-stop-handler  6 • +21 -22
```

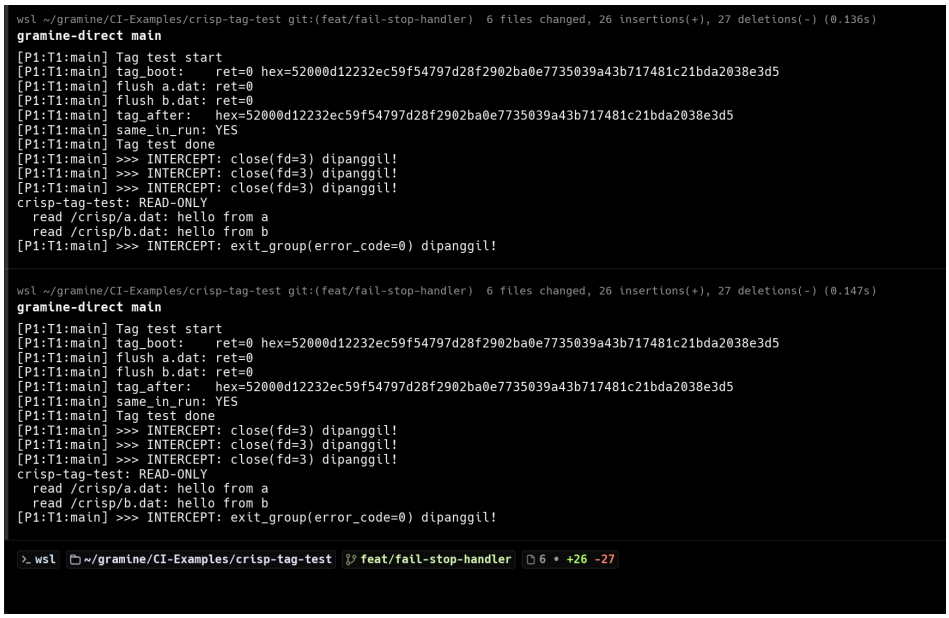
Insight:
2 files existed from the previous test. Tag also successfully computed from two mac metadatas. Flush on a and b succeeded and the compute tag always be deterministic in **one run**.

Test Case #3 - Tag stability (Write and Read) ✅

- **Write**

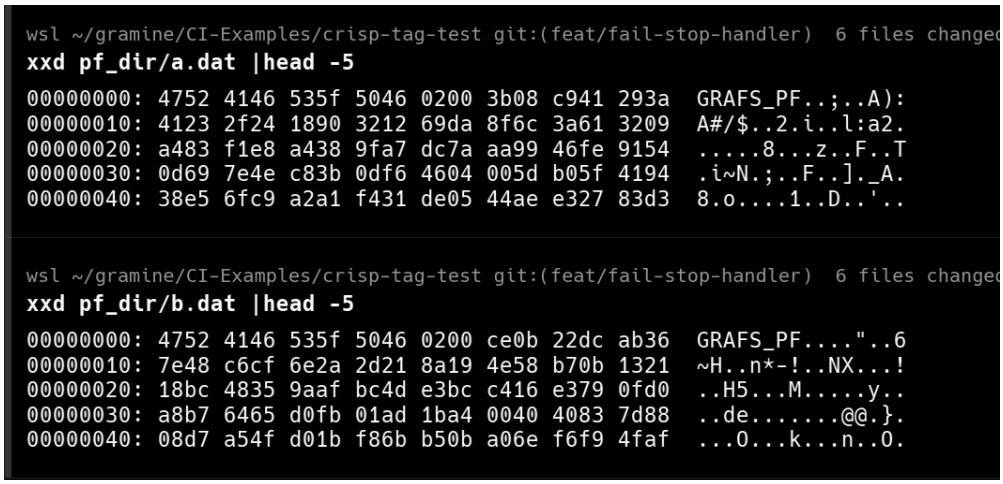
```
wsl ~/gramine/CI-Examples/crisp-tag-test git:(feat/fail-stop-handler)  6 files changed, 21 insertions(+), 22 deletions(-) (0.067s)
diff /tmp/w1.txt /tmp/w2.txt
1c1
< [P1:T1:main] tag_boot:    ret=-1 hex=306cfdf7ff7f00003b89fdf7ff7f00000000000000000000d01d000000000000
---
> [P1:T1:main] tag_boot:    ret=0 hex=68513e07893cf3cbd2b71b631d6c9860b03d78130654780259aeab646519794c

 wsl  ~/gramine/CI-Examples/crisp-tag-test  feat/fail-stop-handler  6 • +21 -22
```

- **Read**



Insight:
Tag changes only when there is write operation. RDONLY on PF wont change MAC.

Test Case #4 - Files in Disk ✅



Insight:
All tracked PF files stored as protected file (encrypted with GRAFS_PF header).

4. Fail-stop handler (MC invariant)

Our assumption and understanding is that when CRISP detect anomaly (rollback, queue timeout, tag mismatch, vault corrupt, invariant violation), proses terminat, not some log warning and silent continue.

> ***Queue timeout*** regulates the tolerance for MC increment latencies and aims to prevent attackers from slowing down MC operations to increment the window of vulnerability. It is tuned according to the write latency of the MC implementation at hand and is especially relevant when the Checker API is not used. When a request waits in the queue longer than the timeout, the runtime will exit prematurely.

Based on the paper, “when a request waits in the queue longer than the timeout, the runtime will exit prematurely”. This also applies on all CRISP unrecoverable failures. Using this fail-stop handler.

Files Related:

- Crisp_init.c
    - __atomic_store_n(&g_crisp.halted, true, __ATOMIC_RELEASE) to make sure that all write before this visible to thread and read with ACQUIRE (related to mc_thread implementation).
    - crisp_wake_all_waiters(). Wake all thread that waits in drain_and_wait. Without this, waiter will block indefinitely becausemc-thread is also inactive.
    - PalProcessExit(1). Terminate all enclave process, not using exit() or abort() due to LIBOS linked with -nostdlib, no libc. So we use PAL primitives.
    - __builtin_unreachable(). The idea is to hint the compiler that the PalProcessExit will not execut. If there is any bug that makes PalProcessExit return (we assume that this is not practically possible), this will not make the compiler emit “"function marked noreturn might return" warning.

To sum up here are the layers:

1. Set halted = true (other threads will see halted via atomic read)
2. Log error
3. Wake waiters (so that it wont stuck on drain_and_wait)
4. Terminate

This implementation will be extensively utilized upon next implementations such as:

- mc-thread where problem spans from queue timeout exceeded, MC increment failed, invariant new_mc == L broken
- close/exit where Flush PF failed, drain_and_wait error
- init (upon startup), where: Vault MC > L (rollback), MC < L (crash), tag mismatch, vault corrupt.

Test Case #1 - Fail-stop halts the process ✅

```
wsl ~/gramine git:(feat/fail-stop-handler)  6 files changed, 51 insertions(+), 52 deletio
cd ~/gramine/CI-Examples/intercept

wsl ~/gramine/CI-Examples/intercept git:(feat/fail-stop-handler)  6 files changed, 51 ins
gramine-direct main
[P1:T1:main] Fail-stop test: halted before = 0
[P1:T1:main] error: CRISP FAIL-STOP: test trigger from harness
[P1:T1:main] crisp_wake_all_waiters
```

Insight:
crisp_fail_stop() actually terminates process with exit code 1, set halted=true atomically, and log error line before exit.

Test Case #2 - Default state without fail-stop ✅

```
wsl ~/gramine/CI-Examples/intercept git:(feat/fail-stop-handler)  6 files c
gramine-direct main
[P1:T1:main] >>> INTERCEPT: close(fd=3) dipanggil!
[P1:T1:main] >>> INTERCEPT: fsync(fd=3) dipanggil!
[P1:T1:main] >>> INTERCEPT: close(fd=3) dipanggil!
hai hai fasilkom
done
[P1:T1:main] >>> INTERCEPT: exit_group(error_code=0) dipanggil!
```

Insight:
crisp_fail_stop() actually terminates process with exit code 1, set halted=true atomically, and log error line before exit.

5. Fsync hook

In this part, we implemented sync between fsync (app thread) and mc-thread. Since the mc-thread is not yet implemented, all the test cases function done are standalone.

Files Related:

- **crisp_fsync.c**
  - crisp_on_fsync. App thread enqueue request, wake mc-thread, and return
  - crisp_drain_and_wait. App thread block until queue drained
  - crisp_wake_all_waiters. Either mc-thread wake all waiter or fail-stop

Note: using queue_mu to protect 4 field in g_crisp which are related to the queue state, like:

- Pending_count. Depicts how many reqs that has been enqueued but not yet proceeded
- Queue_has_work. Hints to mc-thread there is task/work to do
- Batch_in_flight. mc-thread on commit batch
- Oldest_enqueue_us. Timestamp the longest request (for queue timeout)

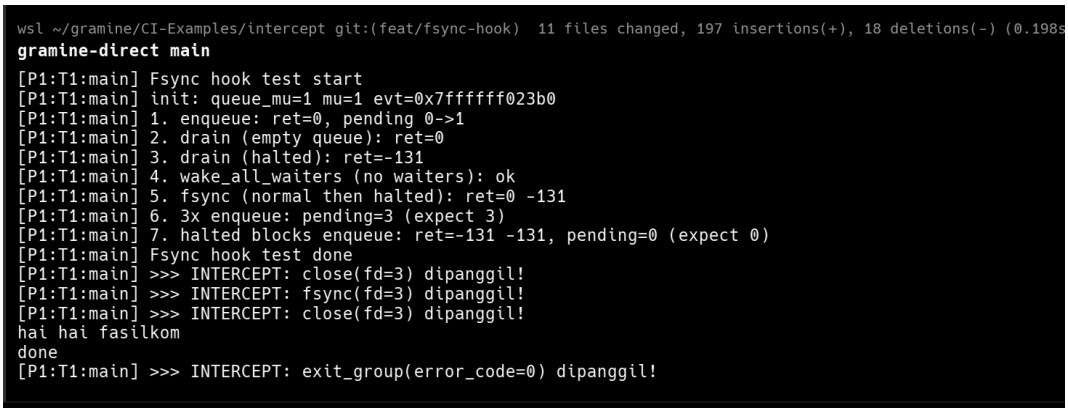
Current CRISP lock ordering hypothesis: queue_mu -> mu -> waiter_lock -> mc_mu

Test Case:

List of test cases:

1. Enqueue counter increment. Calls crisp_on_fsync() once and stores pending_count before and after
2. Drain returns immediately when queue empty.
3. Drain returns ENOTRECOVERABLE when halted, related to set halted=true via atomic RELEASE store, call crisp_drain_and_wait(). Without this fail-stop cannot really stop system.
4. Wake_all_waiters with no waiters present
5. Fsycn normal when halted
6. Multiple enqueue (sequintial 3x)
7. Halt blocks new enqueue

Tests are currently implemented on libos_init.c due to no real syscall integration implemented yet.



Insight:
From the following tests:

1. fsync enqueue. Result: ret=0, pending 0->1
2. drain (empty queue, S=L=0). Result: ret=0, returns immediately
3. drain (halted). Result: ret=-131 (-ENOTRECOVERABLE)
4. wake_all_waiters (no waiters). Result: no crash, doesnt affect state
5. fsync normal and halted. Result: ret=0 then ret=-131
6. 3x consecutive fsync. Result: pending=3
7. halted blocks enqueue. Result: both ret=-131, pending stays 0

-131 is ENOTRECOVERABLE (errno code for halt state)

Based on the result, test confirms expected responses. Confirmed no hang when queue empty, wake_all_waiters can be called without waiters, and no race window where a request gets in after halt.

6. MC-thread

Test Case:
Insight:
From the following tests: